מבוא למדעי המחשב - הרצאה 4

> הערה
>
> **סיכום קצר**
>
> - **הנושא המרכזי:**
> - **הרעיון המרכזי / מה לקחת מההרצאה:**

1. מושגים והגדרות מרכזיים

> חשוב
>
> **הגדרות ותכונות נשמרות**

2. סיכום ההרצאה

נושא 1

> הערה
>
> **הערות והסברים**

Arrays

array is a collection of elements of the same type.

referenced by a common name.

access through an index and operator []

```
int A [4];
```

A has 4 "variables" inside it all of which is an int type.

```
int A [4];

A[0] = 3;
A[1] = 48;
A[2] = 9;
A[3] = 8;
// A[4] is out of bounds
```

same as doing:

```
int A[4] = {3, 48, 9, 8};
```

if there's no index inside the square brackets, the compiler will set the array's size by the number of values inside it:

```
int A[] = {3, 48, 9, 8};
// will be treated as int A[4]
```

fill the array using scanf(),
print the array's values using printf()
one by one

```
int grades[4];

for(int i = 0; i < 4; i++)
    scanf("%d", &grades[i]);

for(int i = 0; i < 4; i++)
    printf("%d ", grades[i]);
```

address calculation

A is the address of the array (here the address of A would be 1000).
the address of the i th element: &A+i * sizeof(array type).
here: 1000 + i * sizeof(int) = 1000 + 4i

if we try to change an array while the index is out of the array's index bounds, the compiler wont throw an error but will just change that specific address with the value its been given.

the program may crash, corrupt memory, or work fine (worst case scenario there will be a kernel panic).

Array size

- array size must be a constant.
  what wont work:

```
int n = 4;
int A[n];
```

```
const int n = 4;
int A[n];
```

what WILL work:

```
#define SIZE 4
int A[SIZE];
// will be A[4] after compilation
```

sizeof(array)

the operator sizeof() returns the size of the array in **bytes**

```
#define SIZE 4
int A[SIZE];
printf("%d", sizeof(A));
// will print 16 since a bit has 4 bytes and there's 4 of it
```

2D Arrays

```
int A[3][4];
```

first index indicates the row position.
second index indicates the column position.

address of element $A[i][j] \ \ in \ \ a \ \ 2D \ \ Array \ \ A[m][m]$:

$$A + ((i \times n) + j) \times sizeof(element)$$

Functions

goes by order:

Declaration
Definition
Invocation

```
int power(int x, int y); // Declaration

int main(){
    // code
    power(x,y); // Invocation
    // more code
    return 0;
}

int power(int x, int y){ // Definition
    //code
    return variable;
}
```

Scope and Lifetime

a scope is a region of the program source code in which the variables name is visible and can be used.
in other words a scope is everything inside a block, everything inside it is staying inside it.

a lifetime is a period of time during program execution in which the variable exists in memory and can be safely accessed.

Global Variables

a variable that is outside of all scopes is called a global variable.
saved in the Data segment
scope: from the line it is defined up to the end of the file
lifetime: during the entire program execution
initialization: by default is 0.

Static Variables

variables declared using the keyword ***static***
save in the Data segment
scope: from its definition until the end of the file
lifetime: during the entire program execution
initialization: unless stated otherwise, default is 0.